

Modélisation et simulation distribuée du mouvement d'un banc de poissons

Rapport du 1er semestre - M1 CHPS

Yohann FRONT-REIGNIER Iram MADANI-FOUATIH Laurence HUANG

22 janvier 2026

Encadrant : Soufian BEN AMOR

Partie 1 - Version séquentielle

Contexte et objectifs

Les 3 règles de Reynolds

Architecture et implémentation

Optimisation et benchmarks

Détails d'implémentation

Partie 2 - Perspectives version parallèle

Motivations

Stratégies de parallélisation

Planning et Livrables S2

Partie 1 - Version séquentielle

- Modélisation d'un **système multi-agents** : un banc de poissons 2D.
- Aucun leader : chaque poisson suit uniquement ses voisins immédiats.
- Ordre global (*flocking*) qui apparaît à partir de règles locales simples.
- Applications : finance, trafic, jeux vidéo, robotique, etc.

Référence : modèle de flocking de Craig Reynolds (1987).

Objectifs du semestre 1

- Implémenter en C++ une simulation réaliste basée sur les **3 règles de Reynolds**.
- Vérifier que le comportement collectif émerge visuellement (banc cohérent).
- Atteindre le temps réel : viser **60 FPS** avec plusieurs centaines de boids.
- Préparer une base propre pour la version **parallèle / distribuée** du semestre 2.

Environ 110 commits répartis entre Yohann, moi et Laurence.

Les 3 règles de Reynolds

1. **Séparation** : éviter les collisions avec les voisins trop proches.
2. **Alignement** : s'adapter à la vitesse moyenne du groupe local.
3. **Cohésion** : se diriger vers le centre de masse des voisins.

Règles purement locales $\Rightarrow$ comportement global émergent.

Formulation mathématique des forces

Pour un boid b de position $\vec{p}_b$ et de vitesse $\vec{v}_b$, avec ensemble de voisins $N(b)$:

Séparation (éviter les collisions) :

$$\vec{F}_{sep} = - \sum_{i \in N(b)} \frac{\vec{p}_b - \vec{p}_i}{\|\vec{p}_b - \vec{p}_i\|^2}$$

Alignement (synchroniser la direction) :

$$\vec{F}_{ali} = \frac{1}{|N(b)|} \sum_{i \in N(b)} \vec{v}_i - \vec{v}_b$$

Cohésion (rejoindre le groupe) :

$$\vec{F}_{coh} = \frac{1}{|N(b)|} \sum_{i \in N(b)} \vec{p}_i - \vec{p}_b$$

Mise à jour : on applique la force totale, on limite par `maxForce`, puis on met à jour la vitesse

Combinaison des 3 règles

À chaque frame, pour chaque boid :

1. On récupère ses voisins dans un **rayon de perception** (30–50 px suivant le paramètre).
2. On calcule les trois forces : $\vec{F}_{sep}$, $\vec{F}_{ali}$, $\vec{F}_{coh}$.
3. On les pondère par les poids de la GUI : w_{sep} , w_{ali} , w_{coh} .
4. On applique la force résultante via `applyForce` puis `update`.

$$\vec{F}_{total} = w_{sep} \vec{F}_{sep} + w_{ali} \vec{F}_{ali} + w_{coh} \vec{F}_{coh}$$

Les poids viennent directement des **sliders SFML**

- **Vector2D** : toutes les opérations vectorielles (position, vitesse, forces).
- **Boid** : état d'un agent + implémentation des règles (séparation, alignement, cohésion).
- **Flock** : contient le vecteur de Boid et une SpatialGrid.
- **SpatialGrid** : grille 2D de cellules stockant des pointeurs sur les boids.
- **GUI (SFML)** : création/suppression de boids, sliders pour les poids et le rayon, affichage.

La méthode centrale est `Flock::updateAll(dt)` qui orchestre une itération complète de la simulation.

Boid :

- Attributs : `position`, `velocity`, `acceleration`, `mass`, `maxSpeed = 30`, `maxForce = 2.5`, `perceptionRadius`.
- Méthodes : `getNeighbors`, `separate`, `align`, `cohesion`, `seek`, `flee`, `update`.

Flock :

- Attributs : `std::vector<Boid> boids`, `SpatialGrid grid`, `neighborRadius`, `sepWeight`, `aliWeight`, `cohWeight`.
- Méthodes : `populateRandom`, `setNeighborRadius`, `setWeights`, `updateAll`.

Recherche de voisins sans grille :

- Pour chaque boid, on parcourt **tous** les autres boids.
- Complexité : $O(n^2)$ comparaisons par itération.

Conséquence pratique (mesurée dans benchmark.cpp, sans affichage) :

- À rayon = 50 px :
 - 100 boids : $\approx 0,19$ ms par itération.
 - 400 boids : $\approx 2,84$ ms.
 - 800 boids : $\approx 10,33$ ms.
- La courbe est clairement **quadratique** quand on augmente n .

Idée : réduire la recherche de voisins à l'environnement local.

- L'espace de simulation est découpé en cellules carrées de taille `cellSize`.
- Chaque cellule contient les pointeurs vers les boids qui s'y trouvent.
- Pour un boid donné, on ne regarde que sa cellule et les 8 voisines.

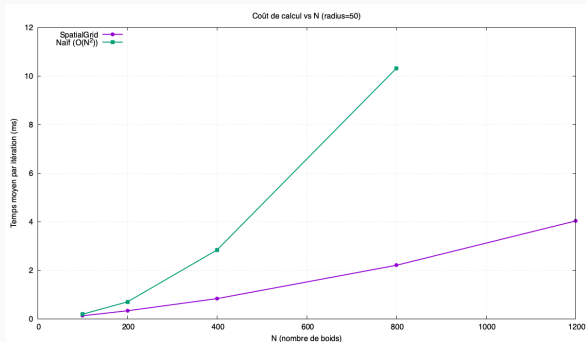
Complexité obtenue :

- Insertion des boids dans la grille : $O(n)$.
- Recherche des voisins : $O(n \cdot k)$ avec k = nombre moyen de boids par cellule (petit).
- En pratique : proche d'un comportement **linéaire** en n .

Benchmark 1 : coût vs rayon (N = 500)

Mesure du temps moyen par itération en faisant varier le **rayon de voisinage**, pour un flock de 500 boids (binaire de benchmark, sans rendu).

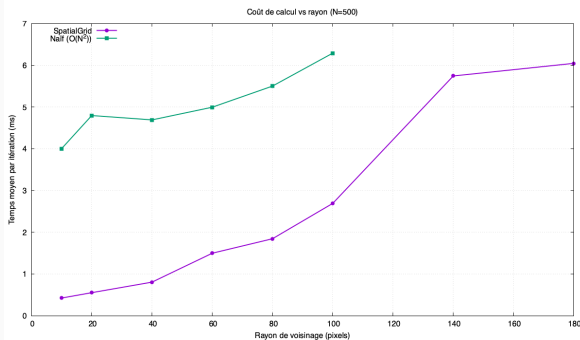
- Naïf : de $\approx 4,0$ ms (rayon 10) à $\approx 6,3$ ms (rayon 100).
- SpatialGrid : de $\approx 0,43$ ms (rayon 10) à $\approx 2,69$ ms (rayon 100), puis $\approx 6,0$ ms pour les très grands rayons.
- La grille reste utilisable même quand le rayon augmente fortement.



Benchmark 2 : coût vs nombre de boids (rayon = 50 px)

Temps moyen par itération en faisant varier **N**, rayon de voisinage fixé à 50 px.

- Naïf :
 - 100 boids : $\approx 0,19$ ms.
 - 400 boids : $\approx 2,84$ ms.
 - 800 boids : $\approx 10,33$ ms.
- SpatialGrid :
 - 100 boids : $\approx 0,13$ ms.
 - 400 boids : $\approx 0,84$ ms.
 - 800 boids : $\approx 2,21$ ms.
 - 1200 boids : $\approx 4,04$ ms.



Configuration	Naïf (ms/it)	Grid (ms/it)
N = 100, rayon = 50	0.19	0.13
N = 400, rayon = 50	2.84	0.84
N = 800, rayon = 50	10.33	2.21
N = 500, rayon = 10	4.01	0.43
N = 500, rayon = 50	4.69	0.81
N = 500, rayon = 100	6.29	2.69

Conclusion :

- La méthode naïve suit bien une croissance $O(n^2)$.
- La grille spatiale a un comportement quasi linéaire, ce qui ouvre la voie à plusieurs milliers de boids en temps réel.

- **Langage** : C++20 (moderne, performant)
- **Build** : CMake 3.20+, GCC/Clang avec flags -O3
- **Graphisme** : SFML 2.6.1 pour visualisation 2D
- **Tests** : Google Test, CTest automatique
- **Documentation** : Doxygen
- **Git** : Gestion de version (97 commits)

- Sliders temps réel : Séparation, Alignement, Cohésion (0-10)
- Rayon de voisinage : 5-200 pixels
- Nombre de boids : 10-500
- Bouton *Apply* pour réinitialiser la simulation
- Affichage des FPS

- **Cohésion** : Agents restent groupés, structure maintenue
- **Alignement** : Direction moyenne émerge rapidement
- **Séparation** : Collisions évitées efficacement
- **Patterns** : Tourbillons, colonnes, vagues observés
- **Performance** : **60 FPS stable** avec 500 boids

Contributions de l'équipe

Yohann

(36 commits)

- Vector2D
- Architecture
- Rapport
- Doc Doxygen

Iram

(38 commits)

- Boid (Règles)
- GUI SFML
- Tests Boid
- Benchmarks

Laurence

(8 commits)

- Flock
- SpatialGrid
- Optimisations

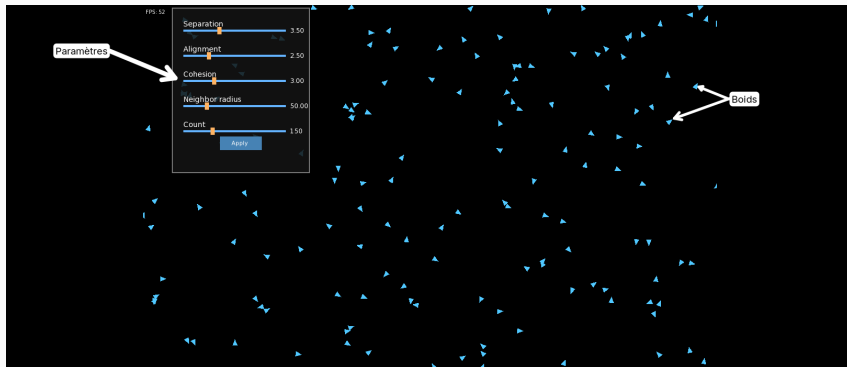
Tests Unitaires (Google Test) :

- `test_vector.bin` : Opérations, magnitude, normalisation
- `test_boid.bin` : Forces, mise à jour, règles Reynolds
- `test_flock.bin` : Gestion collection, SpatialGrid
- `test_ui.bin` : Intégration GUI

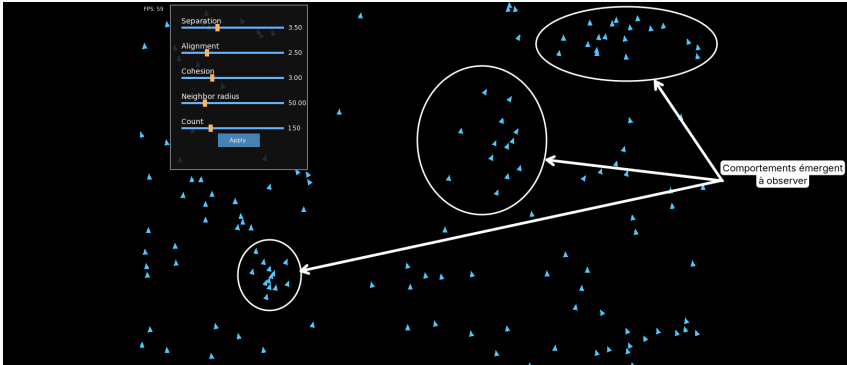
Validation :

- Validation visuelle des 3 règles émergentes
- Tous les tests passent (100% succès)

Affichage du GUI



Affichage du GUI



Partie 2 - Perspectives version parallèle

Pourquoi paralléliser ?

1. Fidélité à la réalité

- Dans la nature : chaque poisson agit **simultanément**
- Séquentiel : traitement un par un $\rightarrow$ s'éloigne de la réalité
- Parallèle/Distribué : calculs simultanés + synchronisation

2. Complexité algorithmique

- Recherche de voisins : $O(n * k)$ par frame (k = voisins/cellule)
- Pour $n = 10000$ boids à 60 FPS : calculs intensifs
- Limites du séquentiel : quelques centaines de boids max

Limites actuelles :

- ~ 500 boids @ 60 FPS
- Architecture séquentielle
- Calcul sur 1 coeur

Objectifs parallélisation :

- **x20 plus d'agents** (10000 boids)
- Maintenir 60 FPS
- Exploiter architectures modernes

Speedup théorique* :

Approche	Gain
OpenMP (8 coeurs)	5-6x
MPI (4 noeuds x 8 coeurs)	15-20x

* Selon loi d'Amdahl avec 90% du code parallélisable

Phase 1 : OpenMP (Mémoire Partagée)

Pourquoi OpenMP ?

- **Simplicité d'utilisation** : directives simples
- **Validation rapide** : tester scalabilité sur une machine

Stratégie de parallélisation

1. Boucle principale

- Indépendance : chaque thread lit les ses voisins
- Race condition : écriture position/vitesse

2. SpatialGrid

- Défi : insertion concurrente
- Alternative : grille locale par thread + fusion finale

Phase 2 : MPI (Mémoire Distribuée)

Approche : Distribution sur plusieurs machines

Décomposition de domaine

- Partitionnement spatial de la zone de simulation
- Chaque processus MPI gère une région
- Boids distribués selon leur position

Communication aux frontières

- Echange des boids proches des frontières
- Synchronisation
- Migration des boids entre processus si changement de région

Défi : Equilibrage de charge + minimiser communications

Phase 3 : Approche Hybride MPI + OpenMP

- **MPI** : Gère la décomposition spatiale (grille distribuée)
 - Chaque processus MPI = une région de l'espace
 - Communication inter-processus pour les frontières
- **OpenMP** : Parallélise les boids au sein de chaque région
 - Threads multiples par processus MPI
 - Traitement parallèle des boids locaux

1. Load balancing

- **Problème** : Distribution inégale des boids
- **Détection** : Mesurer # boids/processus chaque 100 frames
- **Action** : Repartitionnement si écart $> 30\%$
- **Coût** : Communication vs gain calcul

2. Réduction Communications

- **Ghost zone adaptative** : largeur = rayon_max
- **Communication asynchrone** : Envoi et réception par MPI
- **Overlap** : calcul pendant transfert
- **Gain attendu** : 20-30% temps communication

Défi identifiés

1. Overhead communication (MPI)

- Latence réseau : 1-10 ms/message
- Impact si petit nombre de boids/processus
- **Seuil minimal** : ~ 1000 boids pour rentabilité

2. Scalabilité limitée

- Loi d'Amdahl : partie séquentielle (init, render)
- **Efficacité** : 70-80% attendue

3. Load imbalance

- Boids non uniformément répartis
- Repartitionnement coûteux

Phase	Période	Objectifs
Phase 1	Jan - Fév	OpenMP + tests de performances
Phase 2	Fév - Mars	MPI + décomposition domaine
Phase 3	Mars - Avril	Hybride + optimisations
Phase 4	Avril	Benchmarks + analyse
Rendu	~Avril 2025	Rapport

Outils et Environnement

- OpenMP 5.0, MPI 4.0
- Machines multi-coeurs locales
- Profiling : gprof, perf, Intel VTune

Métriques importantes :

1. **Speedup** : $S_p = \frac{T_1}{T_p}$ (temps séquentiel / temps parallèle)
2. **Efficacité** : $E_p = \frac{S_p}{p}$ (speedup / nombre de processeurs)
3. **Strong scaling** : Nombre de boid constant, augmenter le nombre de processeurs
4. **Weak scaling** : Charge par processeur constante, augmenter la taille du problème
5. **FPS maintenu** : Objectif 60 FPS avec 10000 boids

Benchmarks comparatifs : Séquentiel, OpenMP, MPI, Hybride

Bilan semestre 1

- Simulateur complet et fonctionnel (3 règles de Reynolds)
- Architecture modulaire et optimisée (SpatialGrid)
- Visualisation fluide (60 FPS @ 500 agents)

Objectifs Semestre 2

- **S2** : Passage à l'échelle via OpenMP/MPI
- Objectif : 10000+ boids temps réel
- Code source disponible : [lien vers GitHub](#)

Merci de votre attention !

Questions ?

- Reynolds, C. W. (1987). *Flocks, herds and schools : A distributed behavioral model*. SIGGRAPH.
- Vicsek, T. et al. (1995) *Novel type of phase transition in a system of self-driven particles*. *Phys. Rev. Lett.* 75
- Couzin, I. D. et al. (2002) *Collective memory and spatial sorting in animal groups*. *Journal of Theoretical Biology*